Advanced Vision Systems

AGH UST International Course

Tomasz Kryjak, PhD, Mateusz Wąsala, MSc, Hubert Szolc, MSc, Piotr Wzorek, MSc, Krzysztof Błachut, MSc

Tomasz Kryjak
Mateusz Wąsala
Hubert Szolc
Piotr Wzorek
Krzysztof Błachut

Embedded Vision Systems Group
Vision Systems Laboratory
WEAIiIB, AGH University of Krakow

Fourth edition, revised and corrected, Krakow, February 2026

Contents

Block 1

1

Laboratory 1

1. Vision algorithms in Python 3.X – introduction

The exercise will present/mention basic information related to the use of the Python language to perform image and video processing operations and image analysis.

1.1 Programming software

Before starting the exercises, **please create** your own working directory in the place given by the tutor. The name of the directory should be associated with your name – the recommended form is `LastName_FirstName` without special characters.

The Python programming environment – Visual Studio Code (VSCode) – can be used to complete the exercises. Visual Studio Code is a free, lightweight, intuitive and easy to use code editor. It is a universal software for any programming language. Configuration of the editor for exercises comes down only to choosing the appropriate interpreter, or more precisely the appropriate virtual environment in the Python language.

1.2 Python modules used in image processing

Python provides built-in containers (e.g., lists), but it does not offer a native array type designed for efficient vectorised or element-wise operations (such as addition, multiplication, or thresholding of entire data blocks). In practice, the standard solution is to use the *NumPy* module, which provides the `ndarray` type as well as mechanisms such as vectorisation and *broadcasting*. A digital image can then be treated as an $H \times W$ array (grayscale) or $H \times W \times C$ array (color), usually of type `uint8` (range 0–255) or `float` (e.g., 0–1). *NumPy* forms the foundation for most libraries used later for image processing and visualisation.

The most commonly used packages supporting image processing include:

- *SciPy* (the *ndimage* module) – basic filtering and transformation operations (also multidimensional) and *Matplotlib* (*pyplot*) – visualisation of images and plots,
- *Pillow* (a fork of the older, no-longer-maintained PIL module) – convenient loading/saving and simple operations on images,

- *OpenCV* (the *cv2* module) – an extensive set of computer vision algorithms (filters, geometry, features, matching, calibration, etc.),
- *scikit-image* (*skimage*) – a library with image processing algorithms integrated with the Python scientific ecosystem,
- *Plotly* (*plotly*) – interactive plots and visualisations (e.g. heatmaps, 3D plots, dashboards in Jupyter/HTML),
- *Rerun* (*rerun*, *rerun.io*) – logging and interactive visualisation of data (e.g. images, annotations, 3D/robotics) in the Rerun viewer,
- *pandas* – tools for working with tabular data (e.g. loading annotations and metadata from CSV/Excel, analysis of experiment results).

In this course we will mainly rely on *OpenCV*. In parallel we will use *Matplotlib* (especially for displaying results; note: *OpenCV* by default stores colour in BGR order, while *Matplotlib* expects RGB) and occasionally *ndimage* – when a given operation is not available in *OpenCV* or is less convenient to use there.

1.3 Input/output operations

Reading, displaying and saving an image using OpenCV and Matplotlib

Exercise 1.1 Perform a task in which you practice handling files using OpenCV.

1. From the course page, download the image *mandril.jpg* and place it in your own working directory.
2. Load, display, and save an image with the .png extension under a new name using the OpenCV and Matplotlib libraries. Then place a **point** and a **rectangle** on the image and display it on the screen.

R Use:

- OpenCV: `cv2.imread()`, `cv2.imshow()`, `cv2.imwrite()`, `cv2.waitKey()`, `cv2.destroyAllWindows()`, `cv2.rectangle()`.
- Matplotlib: `plt.imread()`, `plt.imsave()`, `plt.figure()`, `plt.imshow()`, `plt.axis()`, `plt.show()` oraz `matplotlib.patches.Rectangle()`.

R The `waitKey()` function displays the image for a specified amount of time (the argument 0 means it waits for a key press). An unnecessary window can be closed with the `destroyWindow()` command with an appropriate parameter, and all open windows can be closed with the `destroyAllWindows()` command. As a good practice, you should always use the `cv2.destroyAllWindows()` function.

■

1.4 Colour space conversion

1.4.1 OpenCV

The function *cvtColor* is used to convert the colour space.

```
IG = cv2.cvtColor(I, cv2.COLOR_BGR2GRAY)
IHSV = cv2.cvtColor(I, cv2.COLOR_BGR2HSV)
```

R Please note that in OpenCV the reading of the image is in **BGR** order, not RGB. This can be important when manually manipulating pixels. For a full list of available

conversions with the relevant formulas check the OpenCV documentation.

Exercise 1.2 Convert the colour space of the given image.
1. Convert the *mandril.jpg* image to greyscale and HSV space. Display the result.
2. Display the H, S, V components of the image after conversion.

■

R Please note that in contrast to e.g. the Matlab package, here the indexing is from 0.

Useful syntax:

```
IH = IHSV[:,:,0]
IS = IHSV[:,:,1]
IV = IHSV[:,:,2]
```

1.4.2 Matplotlib

Here the choice of available conversions is quite limited. Therefore, we need to use the formulas for colour space conversion.

1. RGB to greyscale:

$$G = 0.299 \cdot R + 0.587 \cdot G + 0.144 \cdot B \tag{1.1}$$

The formula can be represented as a function:

```
def rgb2gray(I):
  return 0.299*I[:,:,0] + 0.587*I[:,:,1] + 0.114*I[:,:,2]
```

R The colour map must be set when displaying. Otherwise the image will be displayed in the default, which is not greyscale: `plt.gray()`

2. RGB to HSV.

```
import matplotlib # add at the top of the file
I_HSV = matplotlib.colors.rgb_to_hsv(I)
```

1.5 Scaling, rescaling with OpenCV

Exercise 1.3 Scale the image with *mandril*. Use the *resize* function to scale.

■

Example of use:

```
height, width = I.shape[:2] # retrieving elements 1 and 2, i.e. the corresponding
    height and width
scale = 1.75   # scale factor
Ix2 = cv2.resize(I,(int(scale*height),int(scale*width)))
cv2.imshow("Big Mandril",Ix2)
```

1.6 Arithmetic operations: addition, subtraction, multiplication, difference module

The images are matrices, so the arithmetic operations are quite simple – just like in the Matlab package. Of course, remember to convert to the appropriate data type. Usually double format will be a good choice.

Exercise 1.4 Perform arithmetic operations on the image *lena*.

1. Download the image *lena* from the course page, then load it using a function from OpenCV – add this code snippet to the file that contains the image loader *mandril*. Perform the conversion to greyscale. Add the matrices containing mandril and mena in greyscale. Display the result.
2. Similarly perform the subtraction and multiplication of images.
3. Implement a linear combination of images.
4. An important operation is the module from image difference. It can be done manually – conversion to the appropriate type, subtraction, module (`abs`), conversion to *uint8*. An alternative is to use the function *absdiff* from OpenCV. Please calculate the modulus from the difference of the mandril and lena greyscale images.

■

R When displayed, the image may not visualise correctly, because it is of type *float64* (*double*). In practice, you should:

1. limit the range of values (`clip`),
2. optionally rescale them to the interval `[0,255]`,
3. convert to *uint8*.

Appropriate conversion:

```
img8 = (255*np.clip(img,0,1)).astype(np.uint8) # when values are in [0,1] range
img8 = np.clip(img,0,255).astype(np.uint8) # when values are in [0,255] range
```

1.7 Histogram calculation

Calculating a histogram can be done using the `calcHist` function. Before we move on to that, let us recall the basic control structures in Python: functions and subroutines. Please complete on your own the following function, which computes the histogram of an image in 256 shades of gray:

```
def hist(img):
  h=np.zeros((256,1), np.float32) # creates and zeros single-column arrays
  height, width = img.shape[:2] # shape - we take the first 2 values
    for y in range(height):
...
return h
```

A histogram can be displayed using the `plt.hist` or `plt.plot` functions from the *Matplotlib* library.

The `calcHist` function makes it possible to compute a histogram for several images (or their channels), which is why it takes arrays (e.g. of images) as parameters rather than a single image. However, the following form is used most often:

```
hist = cv2.calcHist([IG],[0],None,[256],[0,256])
# [IG] -- input image
# [0] -- for greyscale images there is only one channel
```

```
# None -- mask (you can count the histogram of a selected part of the image)
# [256] -- number of histogram bins
# [0, 256] -- the range over which the histogram is calculated
```

Please check that the histograms obtained by both methods are the same.

1.8 Histogram equalisation

Histogram equalisation is a popular and important pre-processing operation.

1. Classical histogram equalisation is a global method, it is performed on the whole image. There is a ready-to-use function for this type of equalisation in the OpenCV library:

   ```
   IGE = cv2.equalizeHist(IG)
   ```

2. CLAHE equalisation (Contrast Limited Adaptive Histogram Equalization) – an adaptive method that improves the lighting conditions in an image. Unlike classical histogram equalisation, it operates locally: it equalises the histogram in individual regions of the image rather than for the entire image. The method works as follows:
   - dividing the image into disjoint (square) blocks,
   - computing the histogram in each block,
   - equalising the histogram with a limit on the maximum "height" (the excess is redistributed to neighbouring bins),
   - interpolating pixel values based on the histograms of neighbouring blocks (usually taking into account the four nearest neighbours).

   Details can be found on Wikipedia and in the OpenCV tutorial.

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
# clipLimit - maximum height of the histogram bar - values above are distributed
    among neighbours
# tileGridSize - size of a single image block (local method, operates on separate
    image blocks)
I_CLAHE = clahe.apply(IG)
```

Exercise 1.5 Run and compare both equalisation methods. ■

1.9 Filtration

Filtering is a very important group of operations on images. As an exercise, please run:

- Gaussian filtration (`GaussianBlur`)
- Sobel filtration (`Sobel`)
- Laplacian filter (`Laplacian`)
- Median filtration (`medianBlur`)

R OpenCV documentation – filtration.

Please also note the other functions available:

- bilateral filtration,
- Gabor filters,
- morphological operations.

Exercise 1.6 Run and compare the methods mentioned.

Exercise completion

In order to have the exercise accepted, after completing all the tasks in this laboratory, report your readiness to the teacher. Once it has been approved by the teacher, place the script with the `.py` or `.ipynb` extension in the appropriate location in the course resources on UPeL. ■

2

Laboratory 2

2. Detection of foreground moving objects

What are foreground objects?

These are the objects that are important to us (in the context of the application under consideration). Most often, these are: people, animals, cars (or other vehicles), and luggage (potential bombs). The definition is therefore closely linked to the target application.

Is foreground object segmentation a moving object segmentation?

No. First, an object that has stopped may still be of interest to us (e.g. a person standing in front of a pedestrian crossing). Second, there are many moving elements in the scene that are not relevant to us (e.g. flowing water, a fountain, moving trees or bushes). It is also worth noting that motion is often analysed in image processing. Therefore, the detection of foreground objects can (and should) be supported by the detection of moving objects.

The simplest method of detecting moving objects is based on subtracting consecutive (adjacent) frames. In this exercise, we will implement simple subtraction of two frames, combined with binarisation, connected component labelling, and analysis of the obtained objects. Finally, we will try to treat the result of the subtraction as the outcome of foreground object segmentation and examine the quality of this segmentation.

2.1 Image sequence loading

The sequences used come from a dataset available on the changedetection.net website. In case of problems with accessing the service, the dataset is shared on the UPeL platform. The dataset contains sequences with labels, i.e. each frame of the image has an assigned reference mask of objects (ground truth), in which each pixel belongs to one of five categories: background (0), shadow (50), outside the region of interest (85), unknown motion

(170), and foreground objects (255) – the corresponding greyscale levels are given in parentheses.

For the purposes of this exercise, we are only interested in separating foreground objects from the remaining categories. Additionally, the folder contains a region of interest (ROI) mask and a text file with the time interval for which the results should be analysed (`temporalROI.txt`) – details are provided later in the exercise.

Exercise 2.1 Load the sequences. ■

2.2 Frame subtraction and Binarisation

For the purpose of detecting moving elements, we subtract the previous frame from the current frame; note that in practice we compute the absolute value of this difference. Then, in order to perform binarisation, we need to determine a threshold and choose it so that the objects are relatively clear. Please pay attention to artifacts related to compression.

R To avoid problems with unsigned number subtraction (*uint8*), perform a conversion to type *int* – `IG = IG.astype('int')`.

```
(T, thresh) = cv2.threshold(D,10,255,cv2.THRESH_BINARY)
# D -- input array
# 10 -- threshold value
# 255 -- maximum value to use with the THRESH_BINARY and THRESH_BINARY_INV
    thresholding types
# cv2.THRESH_BINARY -- thresholding type
# T -- our threshold value
# thresh -- output image
```

R The first argument of the returned values by the `cv2.threshold` function is the binarisation threshold (this is useful when using automatic threshold determination, e.g. with the Otsu or triangle method). See the OpenCV documentation for details.

Exercise 2.2 Perform the appropriate operations to obtain a binarised image. ■

2.3 Morphological operations

Morphological operations are simple binary image processing operations (sometimes also used for greyscale images) that modify the shape of objects based on the so-called structuring element (e.g. 3×3).

- erosion (*erosion*) – shrinks objects: makes them smaller, removes small bright noise, can break thin connections,
- dilation (*dilation*) – expands objects: makes them larger, closes small gaps/holes, merges nearby fragments,
- opening (*opening* = erosion → dilation) – removes small objects/noise, smooths the contours,
- closing (*closing* = dilation → erosion) – fills small holes and gaps, connects cracks.

The resulting image is quite noisy. The goal is to obtain as clearly visible a silhouette as possible with minimal disturbances. To improve the result, it is worth adding a median filtering step (before the morphological operations) and, if necessary, adjusting the binarisation threshold.

Exercise 2.3 Perform filtering using erosion and dilation (`erode` and `dilate` from OpenCV).
■

2.4 Labelling and simple analysis

In the next step, the obtained result must be filtered. For this purpose, indexing (assigning labels to groups of connected pixels) will be used, along with the calculation of parameters for these groups. The function connectedComponentsWithStats is used for this. Function call:

```
retval, labels, stats, centroids =  cv2.connectedComponentsWithStats(B)
# retval --  total number of unique labels
# labels -- destination labelled image
# stats -- statistics output for each label, including the background label.
# centroids -- centroid output for each label, including the background label.
```

R When displaying the *labels* image, you need to set the format properly and add scaling.
You can use information about the number of objects found for scaling.

```
cv2.imshow("Labels", np.uint8(labels / retval * 255))
```

Then display the bounding box, area, and index for the largest object. Below is a sample solution to the task. Please run it and, if possible, try to optimise it.

```
I_VIS = I # copy of the input image

if (stats.shape[0] > 1):   # are there any objects

  tab = stats[1:,4] # 4 columns without first element
  pi = np.argmax( tab ) # finding the index of the largest item
  pi = pi + 1 # increment because we want the index in stats, not in tab
  # drawing a bbox
  cv2.rectangle(I_VIS,(stats[pi,0],stats[pi,1]),(stats[pi,0]+stats[pi,2],stats[pi
      ,1]+stats[pi,3]),(255,0,0),2)
  # print information about the field and the number of the largest element
  cv2.putText(I_VIS,"%f" % stats[pi,4],(stats[pi,0],stats[pi,1]),cv2.
      FONT_HERSHEY_SIMPLEX,0.5,(255,0,0))
  cv2.putText(I_VIS,"%d" %pi,(np.int(centroids[pi,0]),np.int(centroids[pi,1])),cv2.
      FONT_HERSHEY_SIMPLEX,1,(255,0,0))
```

Comments on the sample solution:

- `stats.shape[0]` is the number of objects. Since the function also counts the object with index 0 (i.e. background), in the conditional function check if there are objects where the condition is > 1.
- the next two lines are the calculation of the maximum index from column number 4 (field).
- the resulting index should be incremented, as we have omitted element 0 (background) from the analysis.
- We use the `rectangle` function from OpenCV to draw a surrounding rectangle in the image. The syntax `"%f" % stats[pi,4]` allows us to output the value in the appropriate format (f – float). The next parameters are the coordinates of the two opposite vertices of the rectangle. Then the colour in format (B,G,R) and finally the line thickness. See the function documentation for details.

- The function `putText` is used to output text on the image. The coordinates of the lower left vertex are given to determine the position of the text box. The next parameters are the font (full list in the documentation), size and colour.

Exercise 2.4 Use the function `cv2.connectedComponentsWithStats` to label and calculate the parameters of the resulting objects. Display the surrounding rectangle, field and index for the largest object. Based on the tips and examples in the text above, try to optimise the solution. ■

2.5 Evaluation of foreground object detection results

To evaluate a foreground object detection algorithm, the object mask it returns must be compared with a reference mask (the ground truth). The comparison is carried out at the level of individual pixels. If shadows are excluded (as assumed at the outset), four situations are possible:

- TP – *true positive* – a pixel belonging to a foreground object is detected as a pixel belonging to a foreground object,
- TN – *true negative* – a pixel belonging to the background is detected as a pixel belonging to the background,
- FP – *false positive* – a pixel belonging to the background is detected as a pixel belonging to a foreground object,
- FN – *false negative* – a pixel belonging to an object is detected as a pixel belonging to the background.

Based on these values, a number of measures can be calculated. In the following, we will use three: precision (P), recall (R), and the so-called $F1$ measure. These are defined as follows:

$$P = \frac{TP}{TP+FP} \tag{2.1}$$

$$R = \frac{TP}{TP+FN} \tag{2.2}$$

$$F1 = \frac{2PR}{P+R} \tag{2.3}$$

The F1 score is in the range $[0;1]$, with the higher its value, the better result.

Exercise 2.5 Implement the computation of the metrics P, R and $F1$.

R However, we only perform the calculation if a valid reference map is available. To do this, check the dependence of the frame counter and the value from the `temporalROI.txt` file – it must be within the range described there.

■

Exercise 2.6 To summarise, you need to perform moving object detection:

1. Load the `pedestrian` sequence, and then `highway` and `office`.
2. Perform change detection: frame differencing and binarisation.
3. Remove noise (e.g. using a median or Gaussian filter).

4. Apply morphological operations.
5. Perform connected component labelling.
6. Carry out the evaluation.

Exercise completion

In order to have the exercise accepted, after completing all the tasks in this laboratory, report your readiness to the teacher. Once it has been approved by the teacher, place the script with the `.py` or `.ipynb` extension in the appropriate location in the course resources on UPeL. ■

R Information on the following classes.

1. In further exercises we will learn more algorithms and functionalities of the Python language. However, we do not put special emphasis on learning the Python language, but on using in practice the successive algorithms appearing in the lectures. The subject Advanced Vision Systems is not a Python course!
2. The solutions presented should always be considered as examples. The problem can certainly be solved differently, and sometimes better.

2.6 Example results of the algorithm for foreground moving object detection

In order to better verify the different steps of the proposed method for foreground moving object detection, an example result for an image with index 350 is presented.

Figure 2.1: An example of the result of each stage of the algorithm. From left: input image with bounding box, image after binarisation, image after median filtering and morphological operations (erode, dilate), image representing labels after labelling, reference image – ground truth.

3 Laboratory 3

3. Foreground object segmentation

3.1 Main objectives

- familiarisation with the issue of foreground object segmentation and the problems associated with it,
- implementation of simple background modelling algorithms based on a sample buffer – analysis of their advantages and disadvantages,
- implementation of running mean and median approximation algorithms – analysis of their advantages and disadvantages,
- learning about the methods available in *OpenCV* – the Gaussian Mixture Model (also called Mixture of Gaussians) and the method based on the KNN (K-Nearest Neighbours) algorithm,
- familiarisation with the architecture of a neural network and an example application.

3.2 Foreground object segmentation

Foreground object segmentation
Segmentation consists of extracting the objects of interest from the image. This does not have to mean *classification* (e.g. *car*, *pedestrian*); often it is sufficient to know *where* in the image the object is located. The notion of a *foreground object* depends on the application (e.g. in surveillance: people and vehicles).

Background model
In the simplest view, this is an image of an "empty scene", i.e. without foreground objects. In practice, in many methods the background model is implicit (it is not a single image) and cannot be directly displayed (e.g. GMM, ViBE, PBAS).

Background model initialisation
The model has to be initialised at the start of the algorithm. Most often, the first frame (or the first N frames in buffer-based methods) is used as the starting point. Assumption:

in the initialisation phase there are no foreground objects. If objects are present, the model starts with an error, which in the literature is referred to as *bootstrap*. More robust approaches analyse changes over time (and sometimes also spatial coherence), assuming that the background is visible for most of the time.

Background modelling (generation)
A static "background image" works only under strictly controlled conditions. In typical scenes, the background changes (lighting, time of day) or elements of the scene are rearranged (e.g. a chair). Therefore, the background model should adapt; this update process is called background modelling/generation. Lack of adaptation (or adaptation that is too slow) leads to false detections, including *ghosts*.

Pitfalls in background modelling
There is no single method that works well for all scenes. Typical problematic situations include:

- noise and compression artifacts (MJPEG, H.264/265, MPEG),
- camera jitter,
- camera automation (white balance, exposure),
- lighting changes: slow (time of day) and sudden,
- objects present during initialisation,
- moving background elements,
- camouflage (object similar to background),
- objects rearranged in the background.

***Ghost* (in the background model)**
Example: a car stops in a parking lot and after some time becomes incorporated into the background. When it drives away, the "empty spot" may be detected as an object – this is precisely the *ghost* (an object present in the model but not in the current frame).

Update policy
The model update can be:

- **conservative** – only pixels classified as background are updated,
- **liberal** – all pixels are updated.

A conservative policy may "freeze" an error (pixels are never corrected), while a liberal one promotes blending objects into the background and the creation of *ghosts* or streaks.

Shadows
The shadow cast by an object usually meets the criteria of the "foreground", but from the point of view of further analysis it is a disturbance (it changes shape, connects objects). Therefore, a shadow is often treated as a separate class: neither background nor foreground object.

Example
Figure 3.1 shows segmentation using a background model. Note the shadow under the feet of the person on the left and the person behind the barrier (barely visible in the current frame).

3.3 Methods based on frame buffer

The exercise will present two methods: the buffer mean and the buffer median. The buffer size will be assumed to be $N = 60$ samples. In each iteration of the algorithm it is

Figure 3.1: Example of foreground object segmentation using background modelling. a) current frame, b) background model, c) foreground object mask. Source: sequence PETS 2006.

necessary to remove the last frame from the buffer, add the current one (the buffer works on the principle of FIFO queue) and calculate the mean or median from the buffer.

1. First, declare a buffer of size $N \times YY \times XX$(`np.zeros` function), where YY and XX are the height and width of the frame from the *pedestrians* sequence.

```
BUF = np.zeros((YY,XX,N),np.uint8)
```

   The buffer counter (e.g. `iN`) should also be initialised with the value 0. The `iN` parameter acts as a pointer: it indicates the position in the buffer at which the oldest frame should be removed and replaced with the current frame.
2. The buffer should be handled as follows: in a loop, at the position indicated by `iN`, the current greyscale frame should be stored.

```
BUF[:,:,iN] = IG;
```

   Next, the counter `iN` should be incremented and checked to see whether it has reached the buffer size (N). If it has, it must be reset to 0.
3. The calculation of the mean or median is performed using the `mean` or `median` function from the *numpy* library. The dimension (axis) for which the value is to be computed is passed as a parameter (remember that indexing starts from zero). To ensure everything works correctly, the result should be converted to `uint8`.
4. Finally, we perform background subtraction, i.e. we subtract the model from the current frame, and then binarise the result – analogously to the case of the difference between neighbouring frames. Additionally, we apply median filtering to the object mask and/or use morphological operations.
5. Using the code created in the previous exercise, compare the behaviour of the method using the mean and the median. Record the value of the $F1$ score for both cases.

   R Computing the median may take a longer while.

   Consider why the median works better. Test the behaviour on other sequences – in particular on the *office* sequence. Observe the phenomena of smearing and the silhouette blending into the background, as well as the appearance of *ghosts*.

Exercise 3.1 Implement the algorithms (mean and median) based on the above description. ■

3.4 Approximation of mean and median (so-called sigma-delta)

Using a sample buffer requires significant memory resources. The mean can be updated in a simple way (it is worth considering how to update the mean in the buffer without recalculating all elements). In the case of the median, the situation is more difficult: there are no equally fast algorithms for computing it, although it is not necessary to sort all elements at each iteration. Therefore, methods that do not require a buffer are used more often. In the case of approximating the mean, the following relation is used:

$$BG_N = \alpha I_N + (1-\alpha)\, BG_{N-1} \tag{3.1}$$

where: I_N – current frame, BG_N – background model, α – weight parameter (typically 0.01-0.05, although the value depends on the specific application).

The approximation of the median is obtained using the relation:

$$BG_N = \begin{cases} BG_{N-1}+1, & \text{if } BG_{N-1} < I_N, \\ BG_{N-1}-1, & \text{if } BG_{N-1} > I_N, \\ BG_{N-1}, & \text{otherwise.} \end{cases} \tag{3.2}$$

Exercise 3.2 Implement both methods.

1. Take the first frame of the sequence as the first background model. Note the value of the $F1$ indicator for the two methods (set the *alpha* parameter to 0.01). Observe the running time.
2. Experiment with the value of the *alpha* parameter. See how the parameter value affects the background model.

■

R For the running average method, it is crucial that **the background model is stored in a floating-point format** (*float64*).
In formula 3.1, avoiding the use of a loop over the entire image is straightforward. For relation 3.2 this is also possible, but it requires a moment of thought. It is worth consulting the *numpy* documentation for this purpose. Please also note that in Python it is possible to perform arithmetic operations on boolean values.

3.5 Updating policy

So far we have followed a liberal update policy – we have updated everything. We will now try to use a conservative approach – we update only those pixels that have been classified as **background**.

Exercise 3.3 1. For the selected method, implement a conservative update approach. Check its operation.

R Simply remember the previous object mask and use it appropriately in the update procedure.

2. Note the value of the F1 indicator and the background model. Are there any errors in it?

■

3.6 OpenCV – GMM/MOG

In the OpenCV library, one of the most popular methods for foreground object segmentation is available: Gaussian Mixture Models (GMM), also known as Mixture of Gaussians (MoG) – both names are used interchangeably in the literature. In short, the scene is modelled using several Gaussian distributions (e.g. of mean brightness/colour and standard deviation). Each distribution is also described by a weight that reflects how frequently it has been observed in the scene (i.e. its probability of occurrence). The distributions with the highest weights represent the background. Segmentation consists of computing the distance between the current pixel observation and each of the distributions. If the pixel is similar to a distribution recognised as background, it is classified as background; otherwise, it is considered an object. The background model is also updated in a manner similar to equation 3.1. Readers who are interested in more details are referred to the literature, for example to the work by Stauffer and Grimson.

Exercise 3.4 This method is an example of a multivariant algorithm – the background model has several possible representations (variants).

1. Use the `BackgroundSubtractorMOG2` class by creating an object with `create BackgroundSubtractorMOG2`. In a loop, for each image, call the `apply()` method.
2. Experiment with the *history* and *varThreshold* parameters and disable shadow detection. In the `apply()` method you can also set the learning rate – *learningRate*.
3. Compute the $F1$ measure (for the method without shadow detection). Observe how the method works. Note that it is not possible to "display" the background model.

R The OpenCV package provides a version of GMM that is slightly different from the original one – among other things, it includes a shadow detection module and a dynamically changing number of Gaussian components.

■

3.7 OpenCV – KNN

The second method available in OpenCV is KNN algorithm (BackgroundSubtractorKNN).

Exercise 3.5 Please run the KNN method and evaluate it. ■

3.8 Example results of the algorithm for foreground object segmentation

In order to better verify the individual steps of the proposed methods for foreground object segmentation, example results for selected image frames will be presented.

Exercise completion

In order to have the exercise accepted, after completing all the tasks in this laboratory, report your readiness to the teacher. Once it has been approved by the teacher, place the script with the `.py` or `.ipynb` extension in the appropriate location in the course resources on UPeL.

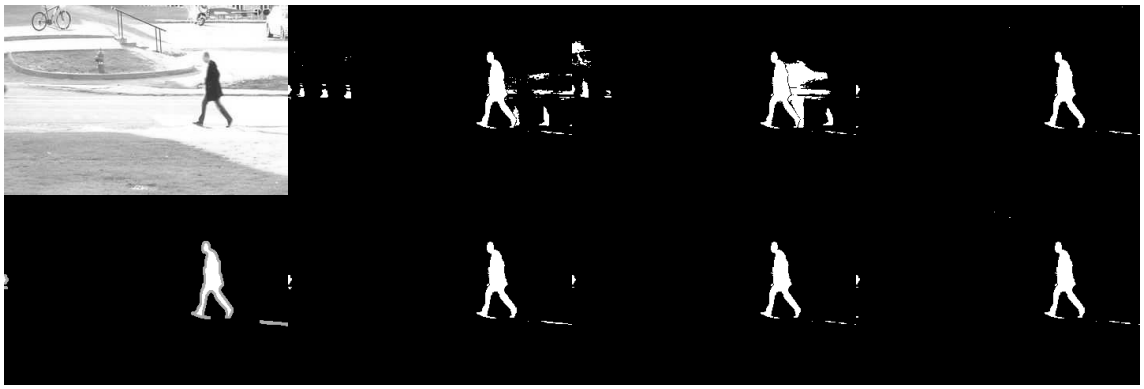

Figure 3.2: Example of the result of the different versions of the algorithm for foreground object segmentation for the *pedestrians* dataset. The frame index is 600. The first column from the left is the input image together with the reference image. The first row is for algorithms using the mean, the second row uses the median. Sequentially from the second column the result of the algorithm is presented: liberal update policy, then the function approximation with a liberal update policy and the function approximation with a conservative update policy.

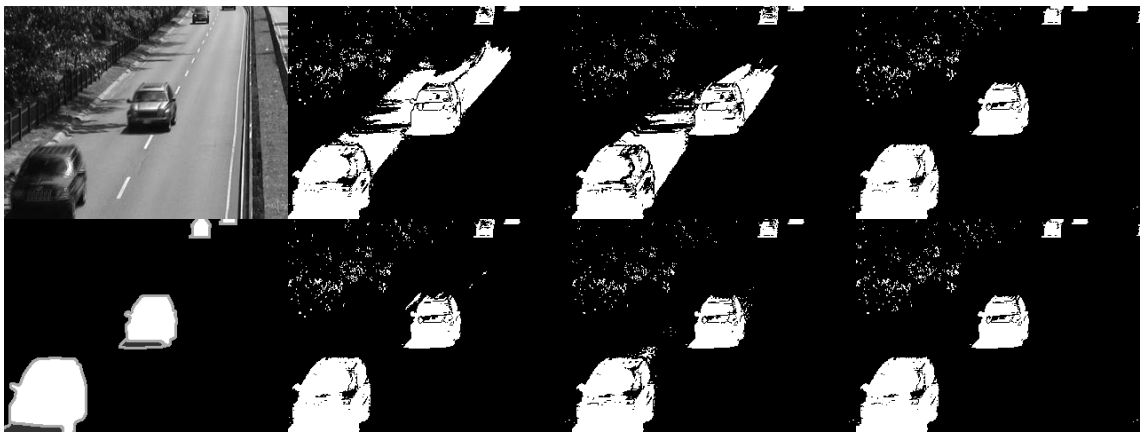

Figure 3.3: Example of the result of the different versions of the algorithm for foreground object segmentation for the *highway* dataset. The frame index is 1200. The first column from the left is the input image together with the reference image. The first row is for algorithms using the mean, the second row uses the median. Sequentially from the second column the result of the algorithm is presented: liberal update policy, then the function approximation with a liberal update policy and the function approximation with a conservative update policy.

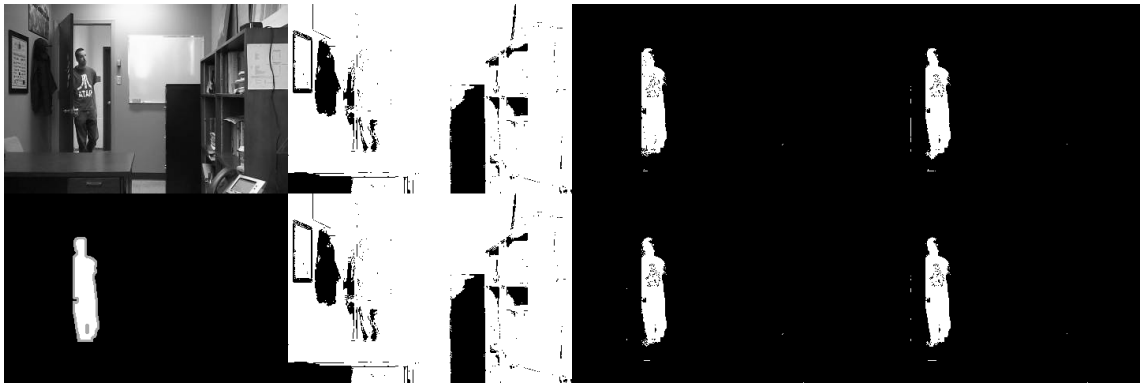

Figure 3.4: Example of the result of the different versions of the algorithm for foreground object segmentation for the *office* dataset. The frame index is 600. The first column from the left is the input image together with the reference image. The first row is for algorithms using the mean, the second row uses the median. Sequentially from the second column the result of the algorithm is presented: liberal update policy, then the function approximation with a liberal update policy and the function approximation with a conservative update policy.

3.9 Using a neural network to segment foreground objects

Neural network architecture-based methods can also be used to segment foreground objects. An example of this is the work[1]. The proposed approach is to use a convolutional neural network called BSUV-Net (Background Subtraction Unseen Videos Net). The architecture is based on a U-net structure with residual connections, which is divided into an encoder and a decoder. The input to network consists of the current frame and two background frames captured at different time scales along with their semantic segmentation maps. The output is a mask containing only foreground objects.

You can find details about this neural network in the article https://arxiv.org/abs/1907.11371. The architecture with sample data and trained network models is made available on Github https://github.com/ozantezcan/BSUV-Net-inference.

Exercise 3.6 **Non-obligatory assignment.** Run the BSUV-Net convolutional neural network.

- Download all the files for the neural network from the UPeL platform.
- The `pedestrians` database will be used, but appropriately scaled to the size of the network input and converted to video sequences.
- Open `infer_config_manualBG.py` file and modify the paths to:
  - in class `SemanticSegmentation` specify the absolute path to the segmentation folder – variable `root_path`,
  - in class `BSUVNet` specify the path to the network model `BSUV-Net-2.0.mdl` in the `trained_models` folder – `model_path`,
  - in class `BSUVNet` specify the path to an image that contains only the background – the first image in the set `pedestrians` – variable `empty_bg_path`.
- In the `inference.py` file specify the path to the network input, i.e. the `pedestrians` video sequence – variable `inp_path` and a path to where the network output is to be stored (path with the file name and file extension) – variable `out_path`.

■

[1] Tezcan, Ozan and Ishwar, Prakash and Konrad, Janusz; BSUV-Net: A Fully-Convolutional Neural Network for Background Subtraction of Unseen Videos, https://arxiv.org/abs/1907.11371.

4

Extra laboratory 1

4. Generalised Hough transform

4.1 Main objectives:

- implementation of the generalised Hough transform,
- pattern search using the generalised Hough transform.

4.2 Implementation of the generalised Hough transform

Exercise 4.1 Pattern searching.

1. From the course page, download the data archive for the exercise and unzip it in your own working directory.
2. Create a new script. Based on the image with the pattern `trybik.jpg`, we will create an array `R-table`. To do this, determine the **contours** and **gradients** in the pattern image.
3. In order to **determine the contour**, one has to first perform image conversion to greyscale and binarisation with an appropriate threshold. Next, use the function `cv2.findContours` to get the contours present in the image.

   R Inverse the values of the image – `cv2.bitwise_not` – before sending it to the function, because it contains a black contour on a white background, and the function `findContours` expects the opposite.

   Note to get lists of all contour points – use the parameter `CHAIN_APPROX_NONE`.

   R In the case where the contour is split, you can use morphological operations to prevent splits or set the `RETR_TREE` parameter, which arranges the contours from the longest to the shortest and allows you to select the longest contour.

4. Display the contour using a function such as `cv2.drawContours`.
5. Compute the gradients using the **Sobel** filters.

- Compute the gradient magnitude,
- Compute the gradient orientation.

It is advisable to normalise the gradient magnitude by its maximum value.

6. **Choose a reference point** – let it be the centre of gravity of the pattern determined from a binarised image of the pattern using moments. The central moments of `m00`, `m10`, and `m01` (function `cv2.moments(bin, 1)`) can be used to determine the centre of gravity.
7. Vectors connecting the contour/contours points to the reference point will be needed to fill the `R-table`. Write into `R-table` **the lengths of these vectors** and **the angles** they form with the OX axis.
   The entry point of the `R-table` is determined by the orientation of the gradient at the contour point (determined from Sobel mask filtering) – **convert radians to degrees** – `R-table` will have 360 rows. We use an accuracy of 1 degree.
   Then, for example, `Rtable[30]` will be a list of polar coordinates of the contour points whose orientation calculated from the gradients is about 30°.
8. Using the image `trybiki2.jpg` and the `R-table` from the previous section, fill the two-dimensional Hough space – recalculate the gradient at each point and for points whose normalised gradient value exceeds 0.5, determine the orientation at that point, then increase the value by 1 in the accumulation space at the points stored in `R-table` according to the dependence:

```
x1 = -r*np.cos(fi) + x
y1 = -r*np.sin(fi) + y
```

   where r,fi – values from the corresponding row in the `R-table`, x,y – coordinates of the point for which the gradient is greater than 0.5.
   The figure of Hough's space is also worth displaying.
9. Search for the maximum in Hough space and mark it on the input image – `trybiki2.jpg`.
10. The result of the operation of the above algorithm is marking the detected maximum on the image and overlaying the template contour around this point. Determine all five maxima in the image.

In order to easily find subsequent maxima, it is advisable to first zero out a certain area around an already detected maximum.

11. An example solution is shown in Fig. 4.1, and additionally Fig. 4.2 illustrates the Hough space.

■

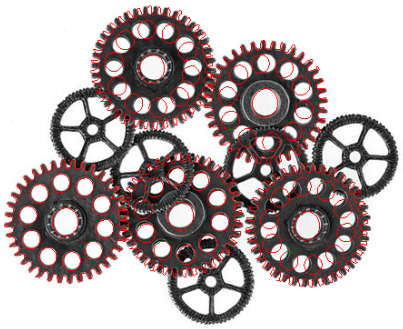

Figure 4.1: Example solution.

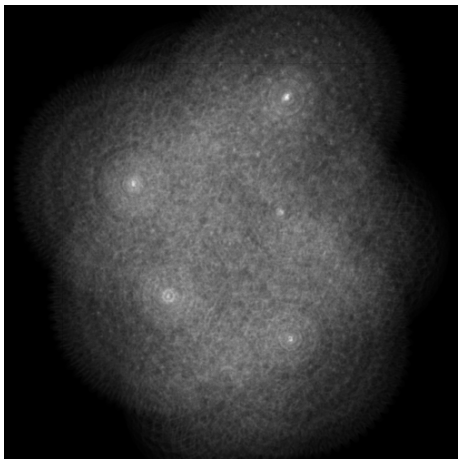

Figure 4.2: Hough space.

5 Mini project 1

5. Mini project 1

Detection of toothbrush bristle defects is carried out mainly using vision systems and machine learning. Data for this type of task includes high-resolution images and scanning electron microscope (SEM) photographs, which makes it possible to analyse the quality of bristle tips.

In production systems (quality control), the most commonly analysed are:

- **Splayed/spread bristles** — deformation of the bristles.
- **Abrasion** — damage to the bristle tips.
- **Errors in fusing/mounting** (*bristle stapling defects*) — incorrect seating of tufts.
- **Improper rounding of tips**.
- **Missing bristles**.

5.1 Project objective

The objective of the project is to develop and test the concept of a vision system for automatic detection of selected toothbrush bristle defects. As part of the project, it is necessary to:

- prepare the data (images) and define defect classes,
- propose and implement a preprocessing stage (e.g. normalisation, denoising, contrast enhancement),
- determine the regions relevant for analysis (e.g. segmentation of bristle tips) or features describing the defects,
- select and train a classification/detection model (or build a rule-based detector using the features),
- evaluate the quality of the solution (e.g. accuracy, precision/recall, confusion matrix) and discuss its limitations.

The dataset is available here: https://drive.google.com/drive/folders/1nXdqY60uZIEWWwLzuxtzRkXygI2xNbS6?usp=sharing.

5.2 Example literature

- Bao, Nengsheng, et al. *A deep learning-based process monitoring system for toothbrush manufacturing defect characterization. Procedia CIRP* **118** (2023): 1072–1077.
- Bergmann, Paul, et al. *MVTec AD–A comprehensive real-world dataset for unsupervised anomaly detection.* In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2019).